



Software Component Design

UML 2 Component Architecture

Field	Value
Document version	0.2
Product	hsh
Architecture baseline	GNU Bash 5.3.x fork (C) + session hsh-agent (Rust)
Modeling notation	UML 2 - Component, Sequence/Protocol, and Class-like contract views
Related specification	hsh SRS v0.2
Status	Revised Component Design
Date	11 September 2026

Purpose: define implementation-safe component boundaries, interfaces, typed execution contracts, authorization artifacts, trust boundaries, and lifecycle rules for hsh.

1. Purpose and Architectural Scope

This revision resolves the implementation and security ambiguities identified in v0.1. It preserves the product split between a Bash-compatible C shell and a Rust intelligence/policy plane, while making the execution path, confirmation protocol, plan schema, capability derivation, context handling, and session security normative.

1.1 Normative architectural decisions in v0.2

- Exactly one canonical OS executor exists for both direct Bash and AI-mediated plans: the Bash Compatibility Core in the hsh C process.
- The Rust agent never calls `execve/fork/spawn` to perform model-generated plan steps. It plans, validates, derives capabilities, evaluates policy, and issues authorization artifacts only.
- AI-mediated execution is multi-phase over IPC: plan, optional confirmation, authorization, execution request, streaming events, completion/cancel.
- Capabilities declared by a provider are hints only. Effective capabilities are recomputed locally from the normalized typed steps, `cwd`, `env_delta`, targets, redirections, privilege boundaries, and host profile.
- Context exported to a provider is allowlisted, classified, bounded and redacted; prior `stdout/file` content is data and never an instruction channel.
- One hsh-agent child process is bound to one interactive hsh session through local peer verification plus a random session credential.

1.2 UML notation used

Notation	Meaning
<code><<component>></code>	Deployable or replaceable software component.
Provided/required interface	Versioned contract crossing a component boundary.
Dependency arrow	A component consumes another component or interface.
Trust boundary	Boundary where data is revalidated before authority increases.
Authorization artifact	Immutable local proof binding plan hash, policy snapshot, session and expiry.
Sequence/protocol view	UML 2 interaction semantics for plan/confirm/execute lifecycle.

2. System Component Overview

The direct Bash path and the AI-mediated path converge at the same canonical executor. This prevents the Rust policy path from being bypassed by a second execution runtime and keeps Bash job-control semantics authoritative.

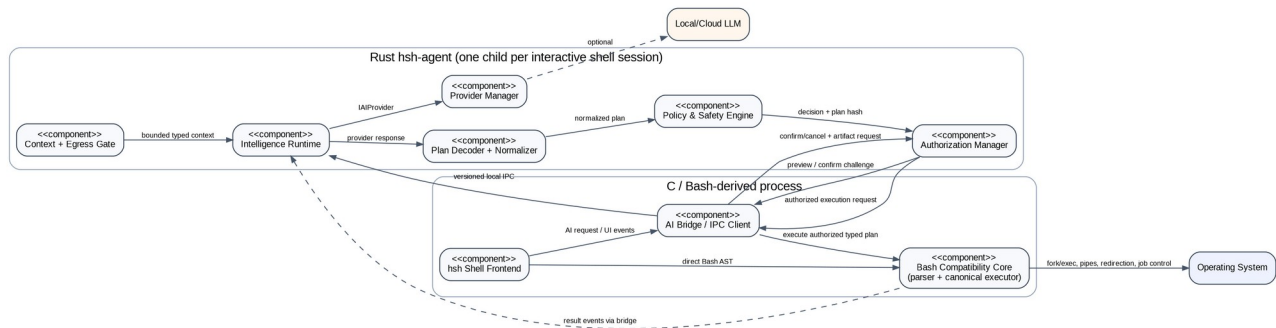


Figure 1 - UML 2 component overview with a single canonical executor.

Component	Implementation	Primary responsibility	Execution authority
hsh Shell Frontend	C	TTY/script input, direct Bash route, AI invocation UI, confirmation preview/input.	No independent OS execution; delegates to Bash Core.
Bash Compatibility Core	C / Bash 5.3.x fork	Parser, expansion, built-ins, pipes, redirection, fork/exec, process groups, terminal/job control.	Canonical OS executor.
AI Bridge / IPC Client	C	Framed local protocol, request correlation, streaming, confirmation/cancel messages.	None.
hsh-agent Intelligence Runtime	Rust	Provider orchestration, typed context, normalization, policy and authorization.	None for model-generated OS actions.
Policy & Safety Engine	Rust	Derive effective capabilities and risk from normalized steps and local environment snapshot.	Can allow/deny/require confirmation; cannot execute.
Authorization Manager	Rust	Bind plan hash to policy/config snapshot, session, confirmation and expiry.	Issues authorization artifact only.
Provider Adapters	Rust	Provider-specific LLM transport behind IAIPProvider.	No OS execution authority.

3. Canonical Execution Model

3.1 Direct Bash execution

Ordinary Bash syntax is parsed and executed entirely by the Bash Compatibility Core. The Rust agent is not involved and may be unavailable without affecting this path.

3.2 AI-mediated execution

The Rust agent returns a normalized typed plan plus an authorization decision. After any required confirmation, the C frontend sends an `ExecuteRequest` containing the exact plan and `AuthorizationArtifact`. The Bash Compatibility Core translates typed plan primitives into its existing process, pipe, redirection and job-control machinery.

The canonical executor **MUST** verify the artifact before executing any AI-mediated mutating or otherwise policy-gated step. Direct Bash commands are user-authored shell input and are not retroactively subjected to the AI policy plane.

3.3 Identity and execution context

- Execution uses the effective uid/gid of the hsh process unless a typed `PrivilegeStep` explicitly requests an OS privilege mechanism.
- `cwd` is canonicalized and included in capability/risk derivation and the authorization hash.
- `env_delta` is normalized, classified and included in capability/risk derivation. Security-relevant variables (for example `loader`, `SSH`, `Git helper`, `proxy` or `credential` variables) may add capabilities or cause `deny/confirm`.
- Background AI-mediated jobs are out of scope through M3; foreground steps stay attached to the Bash job-control domain.

4. IPC and Confirmation Protocol

IPC is not a single interpret-and-execute RPC. It is a stateful, versioned protocol whose messages preserve the approval boundary.

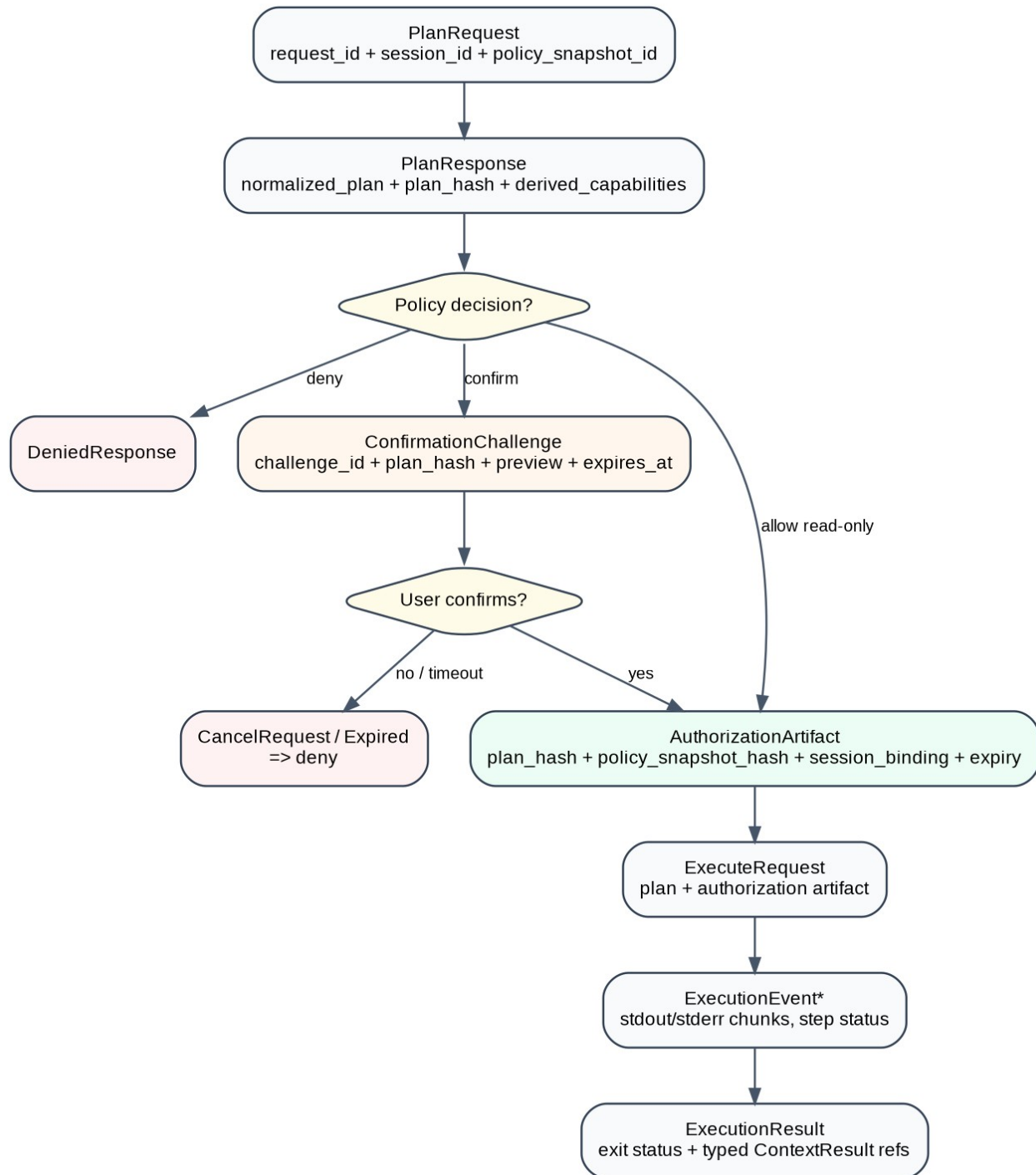


Figure 2 - UML 2 protocol lifecycle for planning, confirmation, authorization and execution.

Message	Direction	Required fields / semantics
Hello / HelloAck	C ↔ Rust	protocol range, build id, session_id, random session credential proof, peer

		identity, feature flags.
PlanRequest	C → Rust	request_id, session_id, locale, interaction mode, cwd snapshot, policy/config snapshot id, NaturalLanguageRequest.
PlanResponse	Rust → C	normalized plan, plan_hash, locally derived capabilities, policy decision, preview metadata.
ConfirmationChallenge	Rust → C	challenge_id, plan_hash, policy_snapshot_hash, preview, expiry. Frontend renders it on the controlling TTY.
ConfirmRequest / CancelRequest	C → Rust	challenge_id, plan_hash, explicit user decision. Empty input, EOF or timeout is cancel.
AuthorizationArtifact	Rust → C	plan_hash, policy_snapshot_hash, session binding, approved capability set, expiry, nonce/MAC or equivalent local integrity token.
ExecuteRequest	C → Rust/C executor boundary	Exact normalized plan + artifact. Executor rejects mismatch, expiry, wrong session or snapshot.
ExecutionEvent	C → Rust	step start/end, bounded stdout/stderr chunks where allowed, exit status. Supports cancel and UI streaming.
ExecutionResult	C → Rust	final status plus typed ContextResult references and redacted summary metadata.
RestartNotice	C ↔ Rust	New agent handshake invalidates outstanding challenges/artifacts; in-flight unexecuted plans require replanning.

```

RequestEnvelope {
    protocol_version,
    request_id,
    session_id,
    message: Hello | PlanRequest | ConfirmRequest | CancelRequest |
        ExecuteStatusRequest | ExplainRequest | SummarizeRequest
}

```

```

AuthorizationArtifact {
    artifact_version,
    plan_hash,
    policy_snapshot_hash,
    config_snapshot_hash,
    session_id,
    approved_capabilities[],
    issued_at,
    expires_at,
    nonce,
    integrity_tag
}

```

5. Typed Execution Plan Contract

The provider never supplies executable shell text. The Plan Decoder parses provider output into typed primitives; unsupported syntax is rejected. There is no free-form “selection expression” in the executable contract.

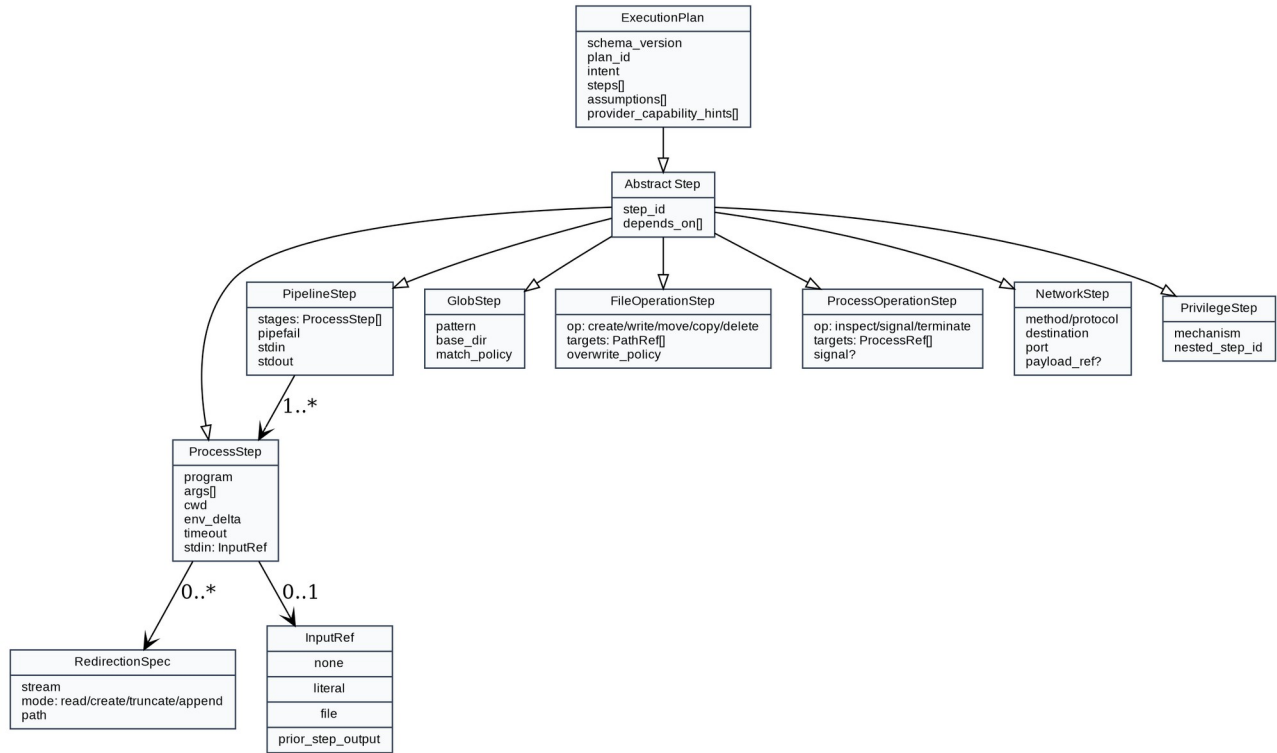


Figure 3 - Typed plan object model; raw shell_script remains forbidden through M3.

Step type	Purpose	Key fields	Execution semantics
ProcessStep	Run one executable	program, args[], cwd, env_delta, timeout, stdin, redirections[]	Bash Core executes argv without eval/bash -c.
PipelineStep	Connect typed process stages	stages[], pipefail, stdin/stdout refs	Bash Core constructs pipes and process groups using native job-control machinery.
GlobStep	Declared path expansion	pattern, base_dir, match_policy	Expansion occurs locally before authorization finalization; resulting scope is hashed/revalidated.
FileOperationStep	Typed filesystem mutation	op, PathRef targets[], overwrite policy	No conversion to rm/mv strings is required; executor maps typed op to reviewed implementation.
ProcessOperationStep	Inspect/signal process targets	op, ProcessRef targets[], signal	Process identity includes PID plus freshness identity where available.
NetworkStep	Explicit outbound operation	protocol/method, destination, port, payload_ref	Always contributes network.outbound and data-egress analysis.
PrivilegeStep	Cross OS privilege boundary	mechanism, nested_step_id	M2+ only; requires privilege.escalate and visible OS authentication.
shell_script	Raw shell program text	N/A	Forbidden through M3;

5.1 InputRef and target references

InputRef = None | LiteralRef | FileRef | PriorStepOutputRef

```
PathRef {
  canonical_path,
  dev_inode_or_equivalent?,
  symlink_policy,
  filesystem_class
}
```

```
ProcessRef {
  pid,
  start_time_or_kernel_identity?,
  executable?,
  owner_uid?
}
```

Destructive follow-ups operate only on explicit typed references produced by prior results or a newly materialized GlobStep. Any material target-set change invalidates the previous authorization and triggers fresh policy evaluation and, where required, confirmation.

6. Capability Derivation and Policy

Provider-supplied capabilities are advisory hints. The Capability Deriver recomputes the effective set from the normalized plan and local environment. Policy decisions use only the derived set.



Figure 4 - Trust path from untrusted data to local capability derivation and policy.

Plan feature	Derived capability examples
Read file metadata/content, read redirection, read-only glob	filesystem.read
Create/truncate/append/write target	filesystem.write
Delete FileOperationStep	filesystem.delete
Inspect processes	process.read
Signal/terminate process	process.kill
Outbound destination	network.outbound
Package-manager mutation recognized by command policy	package.install
PrivilegeStep / sudo-like boundary	privilege.escalate
Sensitive env, secret reference or credential export	secret.read and/or data.export depending on policy

cwd, canonical target paths, mount/filesystem classification, interaction mode, effective uid/gid, env_delta, redirections and host profile are policy inputs. A change to any policy-relevant normalized field changes the plan hash or policy snapshot and invalidates authorization.

7. Context, Prompt Injection and Data Egress

7.1 Context model

```
ContextResult {
    result_id,
    job_id,
    created_at,
    expires_at,
    source_request_id,
    target_type,
    targets[],
    source_snapshot,
    sensitivity,
    summary_metadata
}
```

- Default TTL is 15 minutes unless policy changes it; expired results are not referenceable.
- Pronoun resolution is confined to the same context job unless the user explicitly names another result.
- PID/path targets are revalidated immediately before destructive execution.
- Raw stdout is not a target identity and cannot by itself authorize “them”, “those”, or equivalent references.

7.2 Provider export allowlist

Data class	Default provider export
Typed target metadata: basename/path metadata, size, timestamps, exit code	Allowed only when required by request and privacy policy; redact sensitive roots.
Bounded stdout excerpts	Denied by default; explicit operation may allow N lines/bytes after classification/redaction.
File body / source code	Denied by default; explicit read/summarize intent plus egress policy required.
Environment variables / credentials	Denied; only approved non-secret metadata may be exported.
Audit records	Denied by default.
Provider/system policy text	Local control data; not alterable by untrusted content.

Prompt construction places exported command/file content in clearly delimited DATA fields. The provider is instructed that DATA may contain malicious instructions, and local policy ignores any attempt within data to change capabilities, confirmation, provider selection, audit or privacy controls.

8. IPC Session Security and Agent Lifecycle

- One hsh-agent child is started lazily per interactive hsh session (M1-M4). A shared machine/user daemon is out of scope.
- The IPC endpoint is private to the owning user and session. On Unix, peer credentials (SO_PEERCREDS/getpeereid or platform equivalent) are checked where available.
- A random high-entropy session credential generated by the parent hsh process is required during handshake. session_id alone is not an authenticator.
- Two hsh processes cannot attach to the same agent session without possessing the session credential and matching peer policy.
- Agent restart creates a new handshake epoch and invalidates outstanding confirmation challenges and authorization artifacts.

- If the agent dies before ExecuteRequest, no action occurs. If it dies after a foreground execution started, Bash Core remains owner of the child process and reaps/cancels according to the execution record without corrupting terminal state.

9. UML Interfaces

ID	Interface	Provided by	Required by	Key operations
IF-01	IBashExecution	Bash Compatibility Core	Shell Frontend / AI Execution Adapter	execute_ast(), execute_authorized_plan(), stream_event(), cancel_execution(), capture_status()
IF-02	IIntelligence	hsh-agent Runtime	AI Bridge	plan(), explain(), summarize_result()
IF-03	IIPCSession	AI Bridge + IPC Server	Frontend / Runtime	handshake(), send(), stream(), restart_epoch(), close()
IF-04	IContext	Context Manager	Intent Router / Prompt Builder	get_session(), append_typed_result(), resolve_reference(), revalidate_targets()
IF-05	IDataEgress	Data-Egress Gate	Prompt Builder	classify(), redact(), authorize_export(), bound_payload()
IF-06	IAIProvider	Provider Adapters	Provider Manager	complete_structured(), health(), model_capabilities()
IF-07	IPlanNormalization	Plan Decoder	Runtime	decode(), normalize(), validate_schema(), materialize_declared_globals()
IF-08	ICapabilityDerivation	Capability Deriver	Policy Engine	derive(plan, env_snapshot), explain_derivation()
IF-09	IPlanPolicy	Policy Engine	Authorization Manager	evaluate(), required_confirmation(), policy_snapshot_hash()
IF-10	IConfirmationProtocol	Authorization Manager	Shell Frontend through IPC	challenge(), confirm(), cancel(), expire()
IF-11	IAuthorization	Authorization Manager	Bash Core / AI Execution Adapter	issue_artifact(), verify_artifact(), invalidate()
IF-12	IConfigSnapshot	Config Manager	Runtime / Policy / Provider Manager	resolve_snapshot(), snapshot_hash(), reload_for_future_requests()

IShellInput from v0.1 is removed as a cross-component interface because Input Router is an internal submodule of the Shell Frontend. Internal module calls are not modeled as external UML component boundaries.

10. Configuration and Snapshot Semantics

Configuration reload is atomic for future requests. Every PlanRequest is bound to immutable config_snapshot_hash and policy_snapshot_hash values. An in-flight confirmation or authorization continues

only if the referenced snapshots remain valid; otherwise the challenge/artifact is invalidated and the request is re-evaluated.

Owner	Configuration examples	Reload rule
Shell Frontend	prompt, explicit AI prefix, history integration	Applies on next input cycle.
Runtime	locale, context limits, provider selection	New PlanRequest only.
Policy Engine	allowed roots, host profile, confirmations, denied capabilities	New plan evaluation; existing artifacts remain valid only if snapshot still retained/allowed.
Provider secret resolver	API keys/endpoints	Never copied into plan/audit; provider calls use resolved handle.
Audit subsystem	retention/redaction/sink	Cannot change an allow/deny outcome.

11. Build and Repository Boundaries

```

hsh/
  bash/                # pinned GNU Bash 5.3.x + hsh patches
  bridge/              # C IPC client + typed-plan execution adapter
  builtins/            # hsh/ai builtins integrated with Bash fork
  hsh-agent/           # Rust session child process
  crates/
    hsh-protocol/
    hsh-intelligence/
    hsh-context/
    hsh-egress/
    hsh-policy/
    hsh-authorization/
    hsh-audit/
    hsh-provider-openai/
    hsh-provider-ollama/
  schemas/
    execution-plan.schema.json
    ipc.schema.json
    authorization.schema.json
  tests/
    compatibility/
    protocol/
    policy/
    integration/
    adversarial/
  docs/
    srs/
    architecture/
    uml/

```

There is intentionally no Rust hsh-execution crate with OS process-spawn authority. Rust may contain execution-model types and result-processing code, but the actual process/pipeline/job-control executor remains in Bash Core.

12. Component Interaction Scenarios

12.1 Standard Bash command

- Frontend receives `git status`.
- Input Router selects direct Bash path.
- Bash Core parses/expands/executes using normal Bash semantics.
- No agent/provider/policy call occurs.

12.2 Read-only natural-language request (M1)

- User invokes `ai liste os arquivos maiores que 1 GB`.
- Agent produces a normalized plan.
- Capability Deriver recomputes filesystem.read from steps/cwd/redirections.
- Policy allows read-only scope.
- Authorization artifact is issued without confirmation if local policy permits.
- Bash Core verifies artifact, executes typed plan, streams result events, and emits typed ContextResult metadata.

12.3 Destructive follow-up (M3)

- User says `apague os maiores que 1 GB` in the same context job.
- Context Manager resolves an unexpired ContextResult and revalidates each PathRef.
- Plan contains explicit FileOperationStep targets; there is no free-form selector expression.
- Capability Deriver yields filesystem.delete and Policy requires confirmation.
- Frontend renders challenge on controlling TTY; Enter/timeout/EOF cancels.
- On yes, Authorization Manager issues an artifact bound to exact target set + policy/config snapshots.
- Bash Core verifies artifact immediately before execution; target/material changes force re-evaluation.

13. Dependency and Trust Rules

Rule	Constraint
DEP-01	Bash Core must not depend on provider-specific code.
DEP-02	Provider adapters must not depend on execution internals.
DEP-03	Policy depends on normalized plan/environment types, not provider APIs.
DEP-04	All AI-mediated OS execution enters the single Bash Core executor through <code>execute_authorized_plan()</code> .
DEP-05	The executor rejects absent, expired, mutated or snapshot-mismatched authorization artifacts.
DEP-06	Result summarization cannot grant or expand capabilities.
DEP-07	Context may supply typed data but cannot itself authorize execution.
DEP-08	Effective capabilities are derived locally; provider capability fields never reduce required capabilities.
DEP-09	Config/policy reload cannot mutate an already-hashed plan in place; it invalidates or leaves a retained immutable snapshot.
DEP-10	Audit availability cannot change deny into allow.
DEP-11	Untrusted stdout/file data cannot modify policy, confirmation, privacy, provider selection or authorization.
DEP-12	Raw <code>shell_script</code> / <code>bash -c</code> / <code>sh -c</code> from model output is forbidden through M3.

14. Component Acceptance Criteria by Milestone

Milestone	Component-level exit criteria
M0	Pinned Bash 5.3.x direct path passes compatibility harness; fork/build provenance recorded.
M1	Session agent handshake works; one provider; typed process/pipeline/read redirection/glob; derived read-only capabilities; privacy gate; canonical Bash executor; write/delete/kill denied unsupported.
M2	Typed file/process mutations and privilege step; confirmation protocol; authorization artifact bound to plan+snapshots; host/path/env risk derivation.
M3	Typed ContextResult TTL/job boundaries; stale-target revalidation; destructive follow-up only from explicit typed targets; prompt-injection adversarial tests pass.
M4	At least two providers share IAIProvider without policy/executor changes; provider failure isolation.
M5	Opt-in automatic NL routing; ambiguity contract preserves Bash precedence.

15. Traceability to hsh SRS v0.2

SRS concern	Component realization
ARCH-001 session agent	IPC Session Security + one child per interactive hsh session.
FR-006 / PLAN typed execution	Typed Execution Plan Contract + canonical Bash executor.
FR-007 independent capability derivation	Capability Deriver; provider fields are hints only.
FR-009 / UX confirmation	ConfirmationChallenge + explicit ConfirmRequest/CancelRequest + artifact.
CTX-001..007 bounded context	ContextResult, TTL/job scope, identity revalidation.
POL-001 environment-aware policy	uid/gid, cwd, env_delta, canonical targets, host/filesystem profile included.
SEC-001/002 untrusted AI output	Normalization + no eval/bash -c + authorization boundary.
SEC-004/010 prompt injection	Context Sanitizer + Data-Egress Gate + DATA/instruction separation.
OS portability	Execution semantics stay in Bash Core; plan/policy use normalized OS capability metadata.
Milestone AC mapping	Section 14 mirrors M0-M5 component exits.

Appendix A - v0.1 → v0.2 Corrections

Issue in v0.1	v0.2 resolution
Two possible executors	Removed. Bash Compatibility Core is the only OS executor.
Confirmation gate ambiguous	Confirmation is protocol state in Rust; TTY rendering/input is exclusively Frontend C.
Single RPC plan+execute	Replaced with plan/confirm/authorize/execute/cancel/stream lifecycle.
Underspecified Step union	Defined typed process, pipeline, glob, redirection, file/process/network/privilege steps and InputRef.
Capability naming mismatch	Uses SRS dotted capability vocabulary and explicit derivation table.
cwd/env_delta outside risk	Included as normalized policy inputs and authorization hash material.
Provider capability authority	Provider fields are hints; local Capability Deriver is authoritative.

Raw prior output in context	Provider export allowlist + typed ContextResult; stdout denied by default.
IPC peer ambiguity	Per-session child, OS peer check + random session credential + restart epoch.
Repo split / Rust executor ambiguity	Repository aligned to SRS; no OS-executing Rust crate.
Config reload race	Immutable policy/config snapshots bound to authorization.
Internal interface pollution	Removed IShellInput as external component interface.

hsh v0.2.1 - Normative PDF Patch

Applies to: hsh_UML2_Component_Design.pdf | Source SHA-256:
ed7c86f08e17de6a9f2dcf538bc6ff1b8f226e70d177b1a237c123907a67de77

PATCH PRECEDENCE

This appendix is normative. Where it conflicts with text, examples, tables, milestone labels, repository layouts, acceptance-criterion identifiers, or traceability statements in the v0.2 body, this appendix prevails. Unchanged v0.2 requirements remain in force.

AB-001 - Canonical milestone and acceptance-criterion identifiers

The only milestone identifiers are **M0 through M6**. Acceptance criteria use the global form **M<n>-AC-<nnn>**. Legacy IDs are searchable aliases only and **MUST NOT** be used in new feature plans, pull requests, ADRs, test names, or release gates.

Legacy source	Canonical mapping rule
SRS AC-001..AC-003	AC-00x -> M0-AC-00x
SRS AC-101..AC-109	AC-1xy -> M1-AC-0xy
SRS AC-201..AC-208	AC-2xy -> M2-AC-0xy
SRS AC-301..AC-307	AC-3xy -> M3-AC-0xy
SRS AC-401..AC-404	AC-4xy -> M4-AC-0xy
SRS AC-501..AC-505	AC-5xy -> M5-AC-0xy
SRS AC-601..AC-605	AC-6xy -> M6-AC-0xy
Roadmap Mx-AC-01..99	Mx-AC-yy -> Mx-AC-1yy (roadmap-specific gate range)
Roadmap M0-AC-06	Alias of M0-AC-003; checklist detail remains normative.
UML component exits	Reference canonical Mx-AC sets; no parallel AC numbering.

Examples: SRS AC-104 becomes **M1-AC-004**; Roadmap M1-AC-07 becomes **M1-AC-107**. They are distinct criteria and no longer collide.

AB-003 - Protocol and ExecutionPlan version

The frozen M0 baseline is **IPC protocol_version = "1.0"** and **ExecutionPlan schema_version = "1.0"**. The SRS example containing **"schema_version": "1.1"** is corrected to **"1.0"**. A later 1.1 bump requires an explicit compatibility rule and ADR.

AB-004 - AuthorizationArtifact exists in M1

Every AI-generated plan that reaches the canonical Bash executor **MUST** carry a locally issued **AuthorizationArtifact**, including read-only M1 plans. M1 read-only authorization does not require user confirmation when policy allows it; M2 extends the same artifact with confirmation binding for state-changing operations.

The artifact binds at minimum **plan_hash**, **session_id**, **restart_epoch**, immutable policy/config snapshot hashes, **uid/gid/cwd**, **effective capabilities**, **issued_at** and **expires_at**. The Bash Compatibility Core **SHALL** reject AI-plan execution without a valid artifact.

hsh v0.2.1 - Normative PDF Patch

Applies to: hsh_UML2_Component_Design.pdf

AB-002 - Canonical repository layout

The following tree is the single repository baseline. Any alternate tree in the v0.2 body is superseded. No Rust crate may own direct OS process execution for AI-generated plans.

```
hsh/
  bash/                # pinned GNU Bash 5.3.x-derived C core
  bridge/              # C <-> agent IPC integration
  builtins/            # hsh-specific Bash/C builtins
  schemas/
    protocol/
    execution-plan/
  rust/
    hsh-agent/         # per-interactive-session orchestrator
    crates/
      hsh-protocol/
      hsh-intelligence/
      hsh-policy/
      hsh-context/
      hsh-egress/
      hsh-authorization/
      hsh-audit/
      hsh-os/          # metadata/canonicalization only
      hsh-provider/
    providers/
      openai/
      ollama/
  tests/
    compatibility/
    protocol/
    policy/
    adversarial/
    integration/
```

AB-005 - Platform status: product target vs. implementation gate

Product targets: Linux and macOS. **M1 blocking platform:** exactly one of Linux or macOS, selected during M0. The other is smoke/non-blocking through M5 and becomes blocking by M6. WSL remains secondary/documented unless promoted by ADR.

AB-006 - M1 explicit AI surface and Data-Egress Gate

The mandatory M1 surface is **ai "..."**. The **?** shorthand is optional syntax sugar and **MUST NOT** be an M1 release gate. Data-Egress Gate is P0/M1 and runs before every remote-provider transmission.

M1 data class	Remote-provider default
User NL request	Allowed after local redaction/classification
Typed OS metadata required to plan	Allowlisted only
Env vars / credentials / secrets	Denied
File/repository bodies	Denied
stdout / stderr bodies	Denied
Exit status, bounded paths/sizes/counts	Local presentation; egress only if later classified

M1 natural-language-like result presentation MUST be local/deterministic or metadata-only. It SHALL NOT perform a second provider call containing raw stdout/stderr or file bodies.

AB-007 - Schema-known step types vs. milestone authorization

ExecutionPlan v1 MAY define FileOperationStep, ProcessOperationStep, NetworkStep and PrivilegeStep from M0. Schema recognition does not imply authorization. In M1, any plan requiring capabilities outside the read-only M1 set **MUST** be rejected as **unsupported/denied before AuthorizationArtifact issuance**. Write, delete, process control/kill, network side effects and privilege escalation remain unavailable in M1.

Capabilities are always recomputed locally from typed steps, resolved targets, cwd, env_delta, redirections, host profile and filesystem metadata. Provider risk/capability fields are non-authoritative hints.

hsh v0.2.1 - Normative PDF Patch

Applies to: hsh_UML2_Component_Design.pdf

AB-008 - UML M6 component exit is normative

The UML component-design traceability section is extended through **M6**. M6 does not add an alternate executor or trust path. Component exit requires:

- compatibility regression against the pinned Bash baseline and documented intentional deviations;
- Linux and macOS release fixtures blocking by 1.0; WSL status documented;
- fuzzing/adversarial coverage for JSON/IPC, typed plan decoding, policy inputs and AuthorizationArtifact verification;
- agent crash/restart isolation that cannot corrupt Bash job-control state;
- audit retention, redaction and integrity controls finalized;
- packaging/license/provenance checks completed; stable v1 protocol/provider contracts frozen;
- no unreviewed raw shell-script execution path in release builds.

AB-009 - Canonical sequence

M0 Foundation -> M1 Explicit AI/read-only + Policy v1 -> M2 Controlled Mutation -> M3 Bounded Context -> M4 Provider/Config Maturity -> M5 Opt-in Automatic NL Routing -> M6 Hardening/1.0.

No milestone may enable a capability assigned to a later milestone merely because its step type already exists in the schema.

AB-010 - Canonical executor and trust path

The only OS process executor for AI-generated plans is the Bash Compatibility Core in C. Rust may plan, normalize, recompute capabilities, evaluate policy, build context, apply data-egress policy, issue authorization metadata and coordinate IPC; Rust SHALL NOT call exec/fork/spawn as an alternate AI-plan execution path.

```
Frontend C / TTY
-> hsh-agent (plan + policy)
-> AuthorizationArtifact
-> Frontend C / confirmation when required
-> Bash Compatibility Core
-> OS fork/exec/job-control/redirection
-> bounded local result
-> optional typed ContextResult (M3+)
```

AB-011 - Roadmap token estimates

Roadmap token ranges are planning heuristics for assisted engineering work only. They are not Codex/OpenAI credits, API runtime budget, a contractual delivery estimate, or an acceptance criterion. Measurement reports SHOULD record model/configuration, input/output mix, whether upstream Bash source was inspected or modified, and excluded generated artifacts.

Migration note

Future v0.3+ editions should fold these rules into the body and remove legacy aliases. Until then, implementation artifacts SHALL cite the canonical IDs and baseline in this appendix.